

2.3 服务 (Service) 通信

所属阶段： 第二阶段 · ROS 2 核心通信机制

讲次数量： 8 节

预计时长： ~50 分钟

前置依赖： 2.2 话题 (Topic) 通信

学习目标

完成本节后，学员将能够：

- 解释 ROS 2 Service 的请求/响应 (Request/Response) 通信模型，并与 Topic 的发布/订阅模式对比
- 使用 Python (rclpy) 编写 Service Server 和 Client (非 OOP 与 OOP 两种风格)
- 使用 C++ (rclcpp) 编写 Service Server 和 Client (非 OOP 与 OOP 两种风格)
- 运用 `ros2 service` 系列命令行工具查看、调试和手动调用服务
- 通过 Turtlesim 的 `/spawn` 和 `/set_pen` 服务完成实验，编写 Python Client 在随机位置生成海龟

讲次 1：什么是 ROS 2 Service——请求/响应模式 vs 发布/订阅

知识点

从 Topic 到 Service：何时需要“返回值”

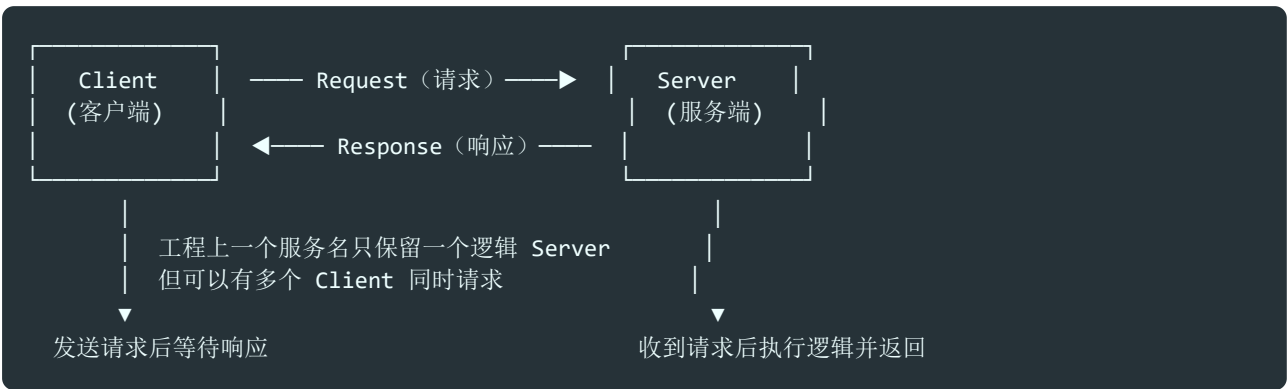
在 2.2 中，我们学习了 Topic 的发布/订阅模式——Publisher 持续发送数据，Subscriber 被动接收，**没有返回值，也不关心对方是否在线**。

这种模式非常适合传感器数据流（激光雷达每秒 10 次扫描）或状态广播（机器人位姿），但并非所有场景都适用：

- 查询机器人当前电量 → 需要**一次请求、一次明确答复**
- 触发相机拍一张照片 → 需要**确认拍照是否成功**
- 在仿真器中生成一只新海龟 → 需要**返回新海龟的名称**

这些场景的共同特征是：**客户端发起请求，服务端处理后返回响应**。ROS 2 用 **Service（服务）** 实现这种通信。

请求/响应模式（Request/Response）



核心特征：

特征	说明
请求/响应语义	每个请求对应一个响应；Client API 可选择同步阻塞或异步回调/Future
双向	数据从 Client 流向 Server (Request)，再从 Server 流回 Client (Response)
单一逻辑提供者	ROS 2 不强制禁止同名 Server，但多个 Server 会让请求路由和结果不可预测，工程上应避免
短暂交互	适合一次性操作，而非持续数据流
有返回值	Response 携带处理结果或状态信息

Service 的三要素

每一个 Service 由三个要素定义：

- 1. **服务名称 (Service Name)**：字符串标识符，如 `/add_two_ints`、`/spawn`
- 2. **服务类型 (Service Type)**：定义 Request 和 Response 的数据结构
- 3. **Server 节点**：处理请求并返回响应的逻辑提供者；同一服务名应只部署一个

服务类型的完整写法：

```
包名/srv/服务名
例：example_interfaces/srv/AddTwoInts
    turtlesim/srv/Spawn
```

在 Python 中导入：`from example_interfaces.srv import AddTwoInts`
在 C++ 中导入：`#include "example_interfaces/srv/add_two_ints.hpp"`

查看服务类型定义

以本课程主要示例 `AddTwoInts` 为例：

```
source /opt/ros/jazzy/setup.bash
ros2 interface show example_interfaces/srv/AddTwoInts
```

预期输出：

```
int64 a
int64 b
---
int64 sum
```

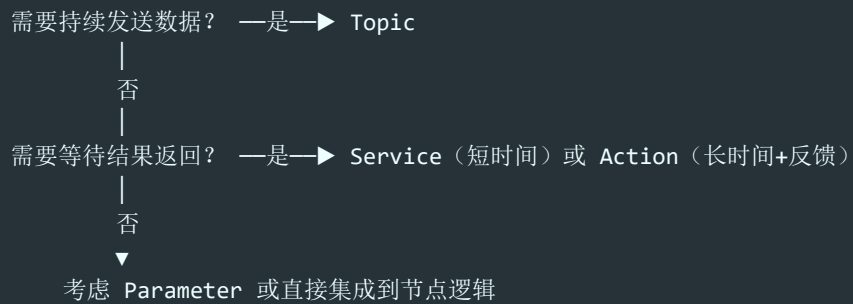
解读：

- `---` 上方是 **Request**（客户端发送的字段）
- `---` 下方是 **Response**（服务端返回的字段）
- 本例：客户端发送两个整数 `a` 和 `b`，服务端返回它们的和 `sum`

Topic vs Service 对比

维度	Topic (话题)	Service (服务)
模式	发布/订阅 (Pub/Sub)	请求/响应 (Req/Res)
方向	单向：Publisher → Subscriber	双向：Client ↔ Server
交互方式	异步数据流，发布后不等待响应	一问一答；Client 可同步或异步等待
提供者数量	多个 Publisher 可发布同一话题	建议每个服务名一个逻辑 Server；框架不强制唯一
返回值	无	有 (Response)
适用场景	传感器流、状态广播、连续控制	查询、触发操作、配置修改

选择通信机制的决策树：



典型场景举例：

场景	推荐机制	原因
激光雷达扫描数据	Topic	持续高频数据流
查询电池电量	Service	一次查询、一次答复
生成仿真海龟	Service	触发操作并返回新海龟名
导航到目标点（需数分钟）	Action	长时间任务，需要进度反馈

Service 在 ROS 2 架构中的位置



Service 经 RMW 映射到底层传输。课程默认的 Fast DDS 路径使用 DDS 请求/响应相关实体完成发现与传输；非 DDS RMW 可以采用不同实现。消息类型支持由 rosidl 生成，开发者通常只使用 rclpy/rclcpp API。

本课程使用的示例服务

服务类型	包	Request 字段	Response 字段	用途
AddTwoInts	example_interfaces	a , b (int64)	sum (int64)	教学示例：两数相加
Spawn	turtlesim	x , y , theta , name	name (string)	在仿真器中生成海龟
SetPen	turtlesim	r , g , b , width , off	(空)	设置海龟画笔颜色

关键点

- Service 基于请求/响应模式，Client 发送 Request，Server 处理后返回 Response
 - 同一服务名应只部署一个逻辑 Server，可有多多个 Client；ROS 2 不会替你阻止同名 Server
 - 服务类型用 包名/srv/服务名 表示，Request 和 Response 字段以 --- 分隔
 - Topic 适合持续数据流；Service 适合一次性查询或触发操作
 - 长时间任务需要进度反馈时，应使用 Action（2.4 讲次）
-

讲次 2：【PY】编写 Service Server

知识点

创建功能包

```
cd ~/ros2_ws/src
ros2 pkg create --build-type ament_python py_srv \
  --dependencies rclpy example_interfaces
```

生成的目录结构：

```
py_srv/
├── package.xml
├── setup.py
├── setup.cfg
├── py_srv/
│   └── __init__.py
└── resource/py_srv
```

Service Server 的核心 API

API	作用
<code>Node.create_service(srv_type, service_name, callback)</code>	创建服务端
<code>callback(request, response)</code>	处理请求并填充响应

回调函数签名：

```
def callback(self, request: AddTwoInts.Request, response: AddTwoInts.Response):
    # 读取 request 字段，计算结果，写入 response 字段
    response.sum = request.a + request.b
    return response # 必须返回 response 对象
```

完整代码：Service Server 节点

创建文件 `~/ros2_ws/src/py_srv/py_srv/add_two_ints_server.py`：

```
#!/usr/bin/env python3
import rclpy
from rclpy.node import Node
from example_interfaces.srv import AddTwoInts

class AddTwoIntsServerNode(Node):
    """提供 add_two_ints 服务，将两个整数相加并返回结果。"""

    def __init__(self):
        super().__init__("add_two_ints_server")
        self.server_ = self.create_service(
            AddTwoInts, "add_two_ints", self.callback_add_two_ints
        )
        self.get_logger().info("Add Two Ints server has been started.")

    def callback_add_two_ints(
        self, request: AddTwoInts.Request, response: AddTwoInts.Response
    ):
        response.sum = request.a + request.b
        self.get_logger().info(
            f"{request.a} + {request.b} = {response.sum}"
        )
        return response

def main(args=None):
    rclpy.init(args=args)
    node = AddTwoIntsServerNode()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()
        rclpy.shutdown()

if __name__ == "__main__":
    main()
```

代码解析

1. 导入服务类型

```
from example_interfaces.srv import AddTwoInts
```

`AddTwoInts` 是服务类型类，同时包含嵌套的 `Request` 和 `Response` 子类。

2. 创建 Service Server

```
self.server_ = self.create_service(
    AddTwoInts,          # 服务类型
    "add_two_ints",      # 服务名（全局为 /add_two_ints）
    self.callback_add_two_ints # 回调函数
)
```

`create_service()` 将节点注册为该服务的 Server。当有 Client 发送请求时，ROS 2 自动调用回调函数。

3. 回调函数处理逻辑

```
def callback_add_two_ints(self, request, response):
    response.sum = request.a + request.b
    return response
```

- `request` 由 ROS 2 自动填充，包含 Client 发送的数据
- 开发者填充 `response` 的字段
- **必须** `return response`，否则 Client 收不到响应

注册入口点并构建

编辑 `setup.py`，添加入口点：

```
entry_points={
    'console_scripts': [
        'add_two_ints_server = py_srv.add_two_ints_server:main',
    ],
},
```

构建并运行：

```
cd ~/ros2_ws
colcon build --packages-select py_srv
source install/setup.bash
ros2 run py_srv add_two_ints_server
```

预期输出：

```
[INFO] [add_two_ints_server]: Add Two Ints server has been started.
```

在另一个终端验证服务已注册：

```
ros2 service list          # 应看到 /add_two_ints
ros2 service type /add_two_ints # 应输出 example_interfaces/srv/AddTwoInts
```

手动调用服务（预告讲次 7 的详细内容）：

```
ros2 service call /add_two_ints example_interfaces/srv/AddTwoInts "{a: 3, b: 5}"
```

Server 终端应打印:

```
[INFO] [add_two_ints_server]: 3 + 5 = 8
```

命令行返回:

```
response:  
example_interfaces.srv.AddTwoInts_Response(sum=8)
```

关键点

- Service Server 通过 `create_service()` 创建, 需提供服务类型、服务名和回调函数
 - 回调函数接收 `request` 和 `response` 两个参数, 填充 `response` 后必须 `return response`
 - 服务名 `"add_two_ints"` 是相对名, 实际注册为 `/add_two_ints`
 - Server 节点需要 `rclpy.spin()` 保持运行, 等待 Client 请求
 - 即使没有 Client 调用, Server 也会正常注册并等待
-

讲次 3：【PY】编写 Service Client (非 OOP 版)

知识点

Service Client 的核心 API

API	作用
<code>Node.create_client(srv_type, service_name)</code>	创建客户端
<code>client.wait_for_service(timeout_sec)</code>	等待 Server 上线
<code>client.call_async(request)</code>	异步发送请求，返回 Future 对象
<code>rclpy.spin_until_future_complete(node, future)</code>	阻塞等待 Future 完成

非 OOP 风格的特点

非 OOP 版本不创建自定义 Node 子类，而是：

1. 直接实例化 `Node("node_name")`
2. 在 `main()` 函数中完成所有逻辑
3. 用 `spin_until_future_complete()` 同步等待响应

适合一次性脚本或简单工具，代码量少、逻辑直观。

完整代码：非 OOP Client

创建文件 `~/ros2_ws/src/py_srv/py_srv/add_two_ints_client_no_oop.py`：

```
#!/usr/bin/env python3
import rclpy
from rclpy.node import Node
from example_interfaces.srv import AddTwoInts

def main(args=None):
    rclpy.init(args=args)
    node = Node("add_two_ints_client_no_oop")

    client = node.create_client(AddTwoInts, "add_two_ints")
    if not client.wait_for_service(timeout_sec=5.0):
        node.get_logger().error("/add_two_ints service is not available")
        node.destroy_node()
        rclpy.shutdown()
        return

    request = AddTwoInts.Request()
    request.a = 3
    request.b = 8

    future = client.call_async(request)
    rclpy.spin_until_future_complete(node, future)

    if future.exception() is not None:
        node.get_logger().error(f"Service call failed: {future.exception()}")
    else:
        response = future.result()
        node.get_logger().info(
            f"{request.a} + {request.b} = {response.sum}"
        )

    node.destroy_node()
    rclpy.shutdown()

if __name__ == "__main__":
    main()
```

代码解析

按完整代码从上到下阅读：

1. `wait_for_service(timeout_sec=5.0)` 给发现阶段设置失败边界；常驻节点若要重连，应使用定时器或状态机退避，而不是在回调中无限循环。
2. `AddTwoInts.Request()` 创建请求，随后填写 `a`、`b` 字段。
3. `call_async()` 立即返回 `Future`；`spin_until_future_complete()` 驱动事件循环并同步等待本次结果。
4. 先检查 `future.exception()`，成功时再读取 `future.result()`；无论哪条路径，最后都销毁节点并关闭 context。

注册入口点并运行

在 `setup.py` 中添加入口点：

```
entry_points={
    'console_scripts': [
        'add_two_ints_server = py_srv.add_two_ints_server:main',
        'add_two_ints_client_no_oop = py_srv.add_two_ints_client_no_oop:main',
    ],
},
```

构建并测试（需要两个终端）：

终端 1 — Server:

```
source ~/ros2_ws/install/setup.bash
ros2 run py_srv add_two_ints_server
```

终端 2 — Client:

```
source ~/ros2_ws/install/setup.bash
ros2 run py_srv add_two_ints_client_no_oop
```

预期 Client 输出:

```
[INFO] [add_two_ints_client_no_oop]: 3 + 8 = 11
```

Server 终端同步打印:

```
[INFO] [add_two_ints_server]: 3 + 8 = 11
```

常见错误排查

现象	原因	解决
提示 Service 不可用后退出	Server 未启动、Domain 不同或发现受阻	检查 Server、 <code>ROS_DOMAIN_ID</code> 和网络后重试
<code>future.result()</code> 抛出异常	服务类型不匹配	检查 Client 和 Server 是否使用相同的 <code>AddTwoInts</code>
无输出就退出	未 <code>source install/setup.bash</code>	重新 source 工作空间

关键点

- 非 OOP Client 直接实例化 `Node()`，在 `main()` 中完成全部逻辑
 - 调用前用 `wait_for_service()` 确保 Server 已上线
 - `call_async()` + `spin_until_future_complete()` 是 Python 同步等待响应的标准组合
 - `AddTwoInts.Request()` 构造请求，`future.result()` 获取响应
 - 非 OOP 风格适合简单脚本；复杂逻辑推荐 OOP 风格（下一讲次）
-

讲次 4: 【PY】编写 Service Client (OOP 版)

知识点

OOP 风格的特点

OOP 版本继承 `Node` 类, 将 Client 逻辑封装在类中:

- 在 `__init__` 中创建 Client
- 用成员方法发送请求
- 用回调函数处理响应 (真正的异步)

适合需要多次调用服务或与其他 ROS 2 组件 (定时器、订阅者) 协同的节点。

完整代码: OOP Client

创建文件 `~/ros2_ws/src/py_srv/py_srv/add_two_ints_client.py` :

```

#!/usr/bin/env python3
import rclpy
from rclpy.node import Node
from example_interfaces.srv import AddTwoInts
from functools import partial

class AddTwoIntsClient(Node):
    """OOP 风格的 AddTwoInts 服务客户端，支持多次异步调用。"""

    def __init__(self):
        super().__init__("add_two_ints_client")
        self.client_ = self.create_client(AddTwoInts, "add_two_ints")
        self.pending_requests_ = 0

    def wait_for_service(self):
        if self.client_.wait_for_service(timeout_sec=5.0):
            return True
        self.get_logger().error("/add_two_ints service is not available")
        return False

    def call_add_two_ints(self, a, b):
        request = AddTwoInts.Request()
        request.a = a
        request.b = b

        future = self.client_.call_async(request)
        self.pending_requests_ += 1
        future.add_done_callback(
            partial(self.callback_call_add_two_ints, request=request)
        )

    def callback_call_add_two_ints(self, future, request):
        try:
            response = future.result()
            self.get_logger().info(
                f"{request.a} + {request.b} = {response.sum}"
            )
        except Exception as exc:
            self.get_logger().error(f"Service call failed: {exc}")
        finally:
            self.pending_requests_ -= 1
            if self.pending_requests_ == 0:
                self.get_logger().info("All requests completed.")
                rclpy.shutdown()

def main(args=None):
    rclpy.init(args=args)
    node = AddTwoIntsClient()
    try:
        if node.wait_for_service():
            node.call_add_two_ints(2, 7)
            node.call_add_two_ints(1, 4)
            node.call_add_two_ints(10, 20)
            rclpy.spin(node)
    except KeyboardInterrupt:

```

```

        pass
    finally:
        node.destroy_node()
        if rclpy.ok():
            rclpy.shutdown()

if __name__ == "__main__":
    main()

```

代码解析

与非 OOP 版相比，这里只有四个新增思想：

1. Client 作为成员变量持久存在，可与订阅、定时器协作。
2. 批量请求前只调用一次 `wait_for_service()`，避免每次发送都重复等待。
3. `add_done_callback()` 异步处理响应；`partial` 额外绑定原始 request，便于日志对应输入。
4. `pending_requests_` 在提交后增加，并在回调 `finally` 中减少；某个 Future 抛异常也不会让节点永远等不到“全部完成”。

非 OOP vs OOP 对比

维度	非 OOP	OOP
Node 创建	<code>Node("name")</code> 直接实例化	继承 <code>Node</code> 类
等待响应	<code>spin_until_future_complete()</code> 阻塞	<code>add_done_callback()</code> 异步回调
多次调用	每次需重新 spin	一次 spin 处理所有回调
代码组织	适合简单脚本	适合复杂节点
与其他组件协同	困难	自然（定时器、订阅者可共存）

注册入口点并运行

在 `setup.py` 中添加入口点：

```

entry_points={
    'console_scripts': [
        'add_two_ints_server = py_srv.add_two_ints_server:main',
        'add_two_ints_client_no_oop = py_srv.add_two_ints_client_no_oop:main',
        'add_two_ints_client = py_srv.add_two_ints_client:main',
    ],
}

```

构建并测试：

```
cd ~/ros2_ws
colcon build --packages-select py_srv
source install/setup.bash
```

终端 1 — Server:

```
ros2 run py_srv add_two_ints_server
```

终端 2 — OOP Client:

```
ros2 run py_srv add_two_ints_client
```

预期 Client 输出（顺序可能因异步而交错）：

```
[INFO] [add_two_ints_client]: 2 + 7 = 9
[INFO] [add_two_ints_client]: 1 + 4 = 5
[INFO] [add_two_ints_client]: 10 + 20 = 30
```

关键点

- OOP Client 继承 `Node`，在 `__init__` 中通过 `create_client()` 创建客户端
 - 用 `call_async()` + `add_done_callback()` 实现真正的异步调用
 - `functools.partial` 用于向回调函数传递额外的 `request` 参数
 - 多次调用后需 `rclpy.spin()` 等待所有回调完成
 - 实际项目中推荐使用 OOP 风格，便于扩展和维护
-

讲次 5：【C++】编写 Service Server

知识点

创建功能包

```
cd ~/ros2_ws/src
ros2 pkg create --build-type ament_cmake cpp_srv \
  --dependencies rclcpp example_interfaces
```

生成的目录结构：

```
cpp_srv/
├── CMakeLists.txt
├── package.xml
├── include/cpp_srv/
└── src/
```

Service Server 的核心 API (C++)

API	作用
<code>create_service<SrvType>(name, callback)</code>	创建服务端
<code>std::bind(&Class::method, this, _1, _2)</code>	绑定成员函数为回调

C++ 回调有两种常见写法：

- **成员函数** + `std::bind`（本课采用，便于扩展）
- **Lambda 表达式**（适合简单逻辑）

完整代码：Service Server 节点

创建文件 `~/ros2_ws/src/cpp_srv/src/add_two_ints_server.cpp`：

```

#include "rclcpp/rclcpp.hpp"
#include "example_interfaces/srv/add_two_ints.hpp"

using std::placeholders::_1;
using std::placeholders::_2;

class AddTwoIntsServerNode : public rclcpp::Node
{
public:
    AddTwoIntsServerNode() : Node("add_two_ints_server")
    {
        server_ = this->create_service<example_interfaces::srv::AddTwoInts>(
            "add_two_ints",
            std::bind(&AddTwoIntsServerNode::callbackAddTwoInts, this, _1, _2));
        RCLCPP_INFO(this->get_logger(), "Add Two Ints server has been started.");
    }

private:
    void callbackAddTwoInts(
        const example_interfaces::srv::AddTwoInts::Request::SharedPtr request,
        example_interfaces::srv::AddTwoInts::Response::SharedPtr response)
    {
        response->sum = request->a + request->b;
        RCLCPP_INFO(this->get_logger(), "%ld + %ld = %ld",
            request->a, request->b, response->sum);
    }

    rclcpp::Service<example_interfaces::srv::AddTwoInts>::SharedPtr server_;
};

int main(int argc, char **argv)
{
    rclcpp::init(argc, argv);
    auto node = std::make_shared<AddTwoIntsServerNode>();
    rclcpp::spin(node);
    rclcpp::shutdown();
    return 0;
}

```

Lambda 写法 (等价替代)

若求简洁, 可用 Lambda 替代成员函数:

```

server_ = this->create_service<example_interfaces::srv::AddTwoInts>(
    "add_two_ints",
    [this](const auto & request, auto response) {
        response->sum = request->a + request->b;
        RCLCPP_INFO(this->get_logger(), "%ld + %ld = %ld",
            request->a, request->b, response->sum);
    });

```

两种写法功能完全等价, 实际项目中按团队风格选择。

配置 CMakeLists.txt

编辑 `~/ros2_ws/src/cpp_srv/CMakeLists.txt` :

```
cmake_minimum_required(VERSION 3.8)
project/cpp_srv

if(CMAKE_COMPILER_IS_GNUCXX OR CMAKE_CXX_COMPILER_ID MATCHES "Clang")
  add_compile_options(-Wall -Wextra -Wpedantic)
endif()

find_package(ament_cmake REQUIRED)
find_package(rclcpp REQUIRED)
find_package(example_interfaces REQUIRED)

add_executable(add_two_ints_server src/add_two_ints_server.cpp)
ament_target_dependencies(add_two_ints_server rclcpp example_interfaces)

install(TARGETS
  add_two_ints_server
  DESTINATION lib/${PROJECT_NAME}
)

ament_package()
```

构建并运行

```
cd ~/ros2_ws
colcon build --packages-select cpp_srv
source install/setup.bash
ros2 run cpp_srv add_two_ints_server
```

验证:

```
ros2 service call /add_two_ints example_interfaces/srv/AddTwoInts "{a: 10, b: 20}"
```

Server 终端输出:

```
[INFO] [add_two_ints_server]: 10 + 20 = 30
```

Python vs C++ Server 对照

操作	Python	C++
导入服务类型	<pre>from example_interfaces.srv import AddTwoInts</pre>	<pre>#include "example_interfaces/srv/add_two_ints.hpp"</pre>
创建 Server	<pre>create_service(AddTwoInts, "name", cb)</pre>	<pre>create_service<AddTwoInts>("name", cb)</pre>
访问请求字段	<pre>request.a</pre>	<pre>request->a</pre>
填充响应字段	<pre>response.sum = ...</pre>	<pre>response->sum = ...</pre>
绑定成员回调	直接传 <code>self.callback</code>	<pre>std::bind(&Class::method, this, _1, _2)</pre>

关键点

- C++ Server 通过 `create_service<SrvType>()` 创建, 回调用 `std::bind` 或 Lambda 绑定
 - 回调参数为 `Request::SharedPtr` 和 `Response::SharedPtr`, 用 `->` 访问字段
 - `ament_target_dependencies` 必须声明 `example_interfaces` 依赖
 - Lambda 和成员函数两种回调风格功能等价, 按项目规范选择
 - C++ Server 同样需要 `rclcpp::spin()` 保持运行
-

讲次 6: 【C++】编写 Service Client (非 OOP / OOP 版)

知识点

本讲次在同一张讲义中展示 C++ Client 的两种风格, 与 Python 讲次 3、4 形成对称。

Service Client 的核心 API (C++)

API	作用
<code>create_client<SrvType>(service_name)</code>	创建客户端
<code>client->wait_for_service(timeout)</code>	等待 Server 上线
<code>client->async_send_request(request)</code>	异步发送请求
<code>rclcpp::spin_until_future_complete(node, future)</code>	阻塞等待 Future 完成

非 OOP 版 Client

创建文件 `~/ros2_ws/src/cpp_srv/src/add_two_ints_client_no_oop.cpp` :

```

#include "rclcpp/rclcpp.hpp"
#include "example_interfaces/srv/add_two_ints.hpp"

using namespace std::chrono_literals;

int main(int argc, char **argv)
{
    rclcpp::init(argc, argv);
    auto node = std::make_shared<rclcpp::Node>("add_two_ints_client_no_oop");

    auto client = node->create_client<example_interfaces::srv::AddTwoInts>("add_two_ints");
    if (!client->wait_for_service(5s))
    {
        RCLCPP_ERROR(node->get_logger(), "/add_two_ints service is not available");
        rclcpp::shutdown();
        return 1;
    }

    auto request = std::make_shared<example_interfaces::srv::AddTwoInts::Request>();
    request->a = 6;
    request->b = 2;

    auto future = client->async_send_request(request);
    if (rclcpp::spin_until_future_complete(node, future, 5s) ==
        rclcpp::FutureReturnCode::SUCCESS)
    {
        auto response = future.get();
        RCLCPP_INFO(node->get_logger(), "%ld + %ld = %ld",
                    request->a, request->b, response->sum);
    }
    else
    {
        RCLCPP_ERROR(node->get_logger(), "Service call timed out or was interrupted");
    }

    rclcpp::shutdown();
    return 0;
}

```

非 OOP 版逻辑与 Python 非 OOP 版完全对称：

1. 创建匿名 `Node`
2. `wait_for_service()` 等待 Server
3. `async_send_request()` 发送请求
4. `spin_until_future_complete()` 阻塞等待
5. `future.get()` 获取响应

OOP 版 Client

创建文件 `~/ros2_ws/src/cpp_srv/src/add_two_ints_client.cpp`：

```

#include "rclcpp/rclcpp.hpp"
#include "example_interfaces/srv/add_two_ints.hpp"
#include <exception>

using namespace std::chrono_literals;
using std::placeholders::_1;

class AddTwoIntsClientNode : public rclcpp::Node
{
public:
    AddTwoIntsClientNode() : Node("add_two_ints_client")
    {
        client_ = this->create_client<example_interfaces::srv::AddTwoInts>("add_two_ints");
        pending_requests_ = 0;
    }

    bool waitForService()
    {
        if (client_->wait_for_service(5s)) {
            return true;
        }
        RCLCPP_ERROR(this->get_logger(), "/add_two_ints service is not available");
        return false;
    }

    void callAddTwoInts(int a, int b)
    {
        auto request = std::make_shared<example_interfaces::srv::AddTwoInts::Request>();
        request->a = a;
        request->b = b;
        pending_requests_++;

        client_->async_send_request(
            request, std::bind(&AddTwoIntsClientNode::callbackCallAddTwoInts, this, _1));
    }

private:
    void callbackCallAddTwoInts(
        rclcpp::Client<example_interfaces::srv::AddTwoInts>::SharedFuture future)
    {
        try {
            auto response = future.get();
            RCLCPP_INFO(this->get_logger(), "Sum: %ld", response->sum);
        } catch (const std::exception & error) {
            RCLCPP_ERROR(this->get_logger(), "Service call failed: %s", error.what());
        }
        pending_requests_--;
        if (pending_requests_ == 0) {
            RCLCPP_INFO(this->get_logger(), "All requests completed.");
            rclcpp::shutdown();
        }
    }

    rclcpp::Client<example_interfaces::srv::AddTwoInts>::SharedPtr client_;
    int pending_requests_;
};

```

```
int main(int argc, char **argv)
{
    rclcpp::init(argc, argv);
    auto node = std::make_shared<AddTwoIntsClientNode>();
    if (!node->waitForService()) {
        rclcpp::shutdown();
        return 1;
    }
    node->callAddTwoInts(10, 5);
    node->callAddTwoInts(12, 7);
    node->callAddTwoInts(1, 2);
    rclcpp::spin(node);
    rclcpp::shutdown();
    return 0;
}
```

OOP 版在 `__init__`（构造函数）中创建 Client，通过 `async_send_request()` 的第二个参数传入回调函数，实现异步响应处理。

更新 CMakeLists.txt

在 `CMakeLists.txt` 中追加两个可执行文件：

```
add_executable(add_two_ints_client_no_oop src/add_two_ints_client_no_oop.cpp)
ament_target_dependencies(add_two_ints_client_no_oop rclcpp example_interfaces)

add_executable(add_two_ints_client src/add_two_ints_client.cpp)
ament_target_dependencies(add_two_ints_client rclcpp example_interfaces)

install(TARGETS
    add_two_ints_server
    add_two_ints_client_no_oop
    add_two_ints_client
    DESTINATION lib/${PROJECT_NAME}
)
```

构建并测试

```
cd ~/ros2_ws
colcon build --packages-select cpp_srv
source install/setup.bash
```

终端 1 — Server:

```
ros2 run cpp_srv add_two_ints_server
```

终端 2 — 非 OOP Client:

```
ros2 run cpp_srv add_two_ints_client_no_oop
```

终端 3 — OOP Client:

```
ros2 run cpp_srv add_two_ints_client
```

四种 Client 风格总览

语言	风格	发送请求	等待响应	适用场景
Python	非 OOP	<code>call_async()</code>	<code>spin_until_future_complete()</code>	简单脚本
Python	OOP	<code>call_async()</code>	<code>add_done_callback()</code>	复杂节点
C++	非 OOP	<code>async_send_request()</code>	<code>spin_until_future_complete()</code>	简单工具
C++	OOP	<code>async_send_request(req, cb)</code>	回调函数	复杂节点

关键点

- C++ 非 OOP Client 用 `async_send_request()` + `spin_until_future_complete()` 同步等待
 - C++ OOP Client 在构造函数中创建 Client, 通过 `std::bind` 注册异步回调
 - C++ 的 Request 用 `std::make_shared<...::Request>()` 创建, 响应用 `future.get()` 获取
 - `wait_for_service(5s)` 使用 chrono 字面量并设置失败边界, 需 `using namespace std::chrono_literals`
 - 实际项目推荐 OOP 风格, 与 Python 侧保持架构一致
-

讲次 7: Service 命令行查看与调试

知识点

ROS 2 提供 `ros2 service` 命令族，无需编写代码即可查看和调用服务。这是开发和调试的重要工具。

前置条件

确保有服务在运行。可以用我们自己的 Server，也可以用 Turtlesim：

```
# 方式 A: 运行课程 Server
ros2 run py_srv add_two_ints_server

# 方式 B: 运行 Turtlesim (内置多个服务)
ros2 run turtlesim turtlesim_node
```

`ros2 service list` — 列出所有服务

```
ros2 service list
```

运行 `add_two_ints_server` 时的输出：

```
/add_two_ints
```

运行 `turtlesim_node` 时的输出（节选）：

```
/spawn
/reset
/turtle1/set_pen
/turtlesim/describe_parameters
...
```

查看服务及其类型（类似 `ros2 topic list -t`）：

```
ros2 service list -t
```

输出示例（Turtlesim）：

```
/reset [turtlesim/srv/Empty]
/spawn [turtlesim/srv/Spawn]
/turtle1/set_pen [turtlesim/srv/SetPen]
...
```

`-t` 参数同时显示服务名和类型，快速了解系统中有哪些服务可用。

ros2 service type — 查看服务类型

```
ros2 service type /add_two_ints
```

输出:

```
example_interfaces/srv/AddTwoInts
```

```
ros2 service type /spawn
```

输出:

```
turtlesim/srv/Spawn
```

进一步查看类型的字段定义:

```
ros2 interface show example_interfaces/srv/AddTwoInts
ros2 interface show turtlesim/srv/Spawn
ros2 interface show turtlesim/srv/SetPen
```

Spawn 服务定义:

```
float32 x
float32 y
float32 theta
string name
---
string name
```

SetPen 服务定义:

```
uint8 r
uint8 g
uint8 b
uint8 width
uint8 off
---
```

注意 SetPen 的 Response 为空 (--- 下方无字段), 表示"设置成功即可, 无需返回数据"。

ros2 service call — 手动调用服务

语法:

```
ros2 service call <服务名> <服务类型> "<YAML格式的请求数据>"
```

示例 1: 调用 AddTwoInts

```
ros2 service call /add_two_ints example_interfaces/srv/AddTwoInts "{a: 7, b: 13}"
```

返回:

```
requester: making request: example_interfaces.srv.AddTwoInts_Request(a=7, b=13)

response:
example_interfaces.srv.AddTwoInts_Response(sum=20)
```

示例 2: 生成一只新海龟

```
ros2 service call /spawn turtlesim/srv/Spawn "{x: 4.0, y: 2.0, theta: 1.57, name: 'my_turtle'}"
```

返回:

```
response:
turtlesim.srv.Spawn_Response(name='my_turtle')
```

若 `name` 留空, Turtlesim 自动分配名称 (如 `turtle2`) :

```
ros2 service call /spawn turtlesim/srv/Spawn "{x: 8.0, y: 8.0, theta: 0.0, name: ''}"
```

示例 3: 设置海龟画笔为红色

```
ros2 service call /turtle1/set_pen turtlesim/srv/SetPen "{r: 255, g: 0, b: 0, width: 3, 'off': 0}"
```

参数说明:

字段	含义
<code>r, g, b</code>	RGB 颜色值 (0-255)
<code>width</code>	画笔宽度 (像素)
<code>off</code>	<code>1</code> = 抬起画笔 (不画线) , <code>0</code> = 放下画笔

`ros2 service find` — 按类型查找服务

当你知道服务类型但不知道服务名时:

```
ros2 service find example_interfaces/srv/AddTwoInts
```

输出：

```
/add_two_ints
```

```
ros2 service find turtlesim/srv/Spawn
```

输出：

```
/spawn
```

这在大型系统中特别有用——比如你想调用 `Spawn` 服务，但不确定具体的服务名。

调试工作流

推荐的 Service 调试流程：

1. `ros2 service list -t`

→ 发现有哪些服务
2. `ros2 interface show <type>`

→ 了解 Request/Response 字段
3. `ros2 service call ...`

→ 手动测试服务功能
4. 编写 Client 代码

→ 自动化调用
5. `ros2 service find <type>`

→ 验证你的 Client 是否连对了服务

与 `ros2 topic` 命令的对称关系

操作	Topic 命令	Service 命令
列出所有	<code>ros2 topic list</code>	<code>ros2 service list</code>
列出含类型	<code>ros2 topic list -t</code>	<code>ros2 service list -t</code>
查看类型	<code>ros2 topic type <topic></code>	<code>ros2 service type <service></code>
手动发送	<code>ros2 topic pub ...</code>	<code>ros2 service call ...</code>
查看数据	<code>ros2 topic echo ...</code>	(无直接等价, 用 <code>service call</code> 看响应)
按类型查找	(无)	<code>ros2 service find <type></code>

关键点

- `ros2 service list` 列出活跃服务，`-t` 同时显示类型
- `ros2 service type` 查看服务类型，`ros2 interface show` 查看字段定义
- `ros2 service call` 手动调用服务，请求数据用 YAML 格式
- `ros2 service find` 按类型反查服务名
- 先命令行验证服务行为，再编写 Client 代码，可大幅减少调试时间

讲次 8：【LAB】用 Turtlesim 实验 Service 通信

知识点

本实验综合运用本节课所学，在 Turtlesim 仿真环境中完成三个任务：

1. 用命令行发现并调用 Turtlesim 服务
2. 用 `/spawn` 生成海龟，用 `/set_pen` 改变画笔颜色
3. 编写 Python Client，在随机位置自动生成海龟

实验准备

终端 1 — 启动 Turtlesim：

```
source /opt/ros/jazzy/setup.bash
ros2 run turtlesim turtlesim_node
```

任务 1：发现 Turtlesim 的服务

```
ros2 service list
```

找到与业务相关的服务：

服务名	类型	功能
<code>/spawn</code>	<code>turtlesim/srv/Spawn</code>	在指定位置生成新海龟
<code>/reset</code>	<code>turtlesim/srv/Empty</code>	重置仿真器
<code>/turtle1/set_pen</code>	<code>turtlesim/srv/SetPen</code>	设置 turtle1 画笔颜色
<code>/turtle1/teleport_absolute</code>	<code>turtlesim/srv/TeleportAbsolute</code>	瞬移海龟到指定位置
<code>/turtle1/teleport_relative</code>	<code>turtlesim/srv/TeleportRelative</code>	相对当前位置瞬移

查看 `Spawn` 服务定义：

```
ros2 interface show turtlesim/srv/Spawn
```

任务 2：命令行生成海龟并改变画笔颜色

生成一只红色海龟：

```
# 在 (5.0, 5.0) 位置生成海龟，使用自定义名称避免后续冲突
ros2 service call /spawn turtlesim/srv/Spawn "{x: 5.0, y: 5.0, theta: 0.0, name: 'my_turtle'}"
```

Turtlesim 窗口中应出现第二只海龟。

将 turtle1 的画笔设为绿色：

```
ros2 service call /turtle1/set_pen turtlesim/srv/SetPen "{r: 0, g: 255, b: 0, width: 4, 'off': 0}"
```

让 turtle1 向前移动以画一条线验证颜色变化：

```
# --rate 10 表示以 10 Hz 持续发布，按 Ctrl+C 停止
ros2 topic pub -r 10 /turtle1/cmd_vel geometry_msgs/msg/Twist \
  "{linear: {x: 2.0, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 0.0}}"
# 让海龟走一段后按 Ctrl+C 停止，观察轨迹是否为绿色
```

注意： `--once` 只发一条消息，海龟移动距离极短（约 0.03 像素），几乎看不到轨迹。要看到明显的画线效果，必须持续发送速度指令。

将 my_turtle 的画笔设为蓝色：

```
ros2 service call /my_turtle/set_pen turtlesim/srv/SetPen "{r: 0, g: 0, b: 255, width: 4, 'off': 0}"
```

注意：新生成的海龟也有自己的 `set_pen` 服务，服务名格式为 `/<turtle_name>/set_pen`。

任务 3：编写 Python Client 随机生成海龟

创建功能包（若已有 `py_srv` 可复用）：

```
cd ~/ros2_ws/src
ros2 pkg create --build-type ament_python turtle_spawn \
  --dependencies rclpy turtlesim
```

创建文件 `~/ros2_ws/src/turtle_spawn/turtle_spawn/random_spawner.py`：

```

#!/usr/bin/env python3
import random
import math

import rclpy
from rclpy.node import Node
from turtlesim.srv import Spawn

class RandomSpawnerNode(Node):
    """每隔 2 秒在随机位置调用 /spawn 服务生成一只新海龟。"""

    def __init__(self):
        super().__init__("random_spawner")
        self.counter_ = 0
        self.spawn_client_ = self.create_client(Spawn, "/spawn")
        self.timer_ = self.create_timer(2.0, self.spawn_random_turtle)
        self.get_logger().info("Random spawner started, spawning every 2 seconds")

    def spawn_random_turtle(self):
        self.counter_ += 1
        name = f"turtle{self.counter_ + 1}" # turtle1 已存在, 从 turtle2 开始

        x = random.uniform(0.5, 10.5)
        y = random.uniform(0.5, 10.5)
        theta = random.uniform(0.0, 2.0 * math.pi)

        if not self.spawn_client_.wait_for_service(0.1):
            self.get_logger().warn("Waiting for /spawn service...")
            return

        request = Spawn.Request()
        request.x = x
        request.y = y
        request.theta = theta
        request.name = name

        future = self.spawn_client_.call_async(request)
        future.add_done_callback(
            lambda f: self.spawn_callback(f, name, x, y)
        )

    def spawn_callback(self, future, name, x, y):
        response = future.result()
        if response.name:
            self.get_logger().info(
                f"Spawned '{response.name}' at ({x:.1f}, {y:.1f})"
            )
        else:
            self.get_logger().error(f"Failed to spawn '{name}'")

def main(args=None):
    rclpy.init(args=args)
    node = RandomSpawnerNode()
    try:
        rclpy.spin(node)

```

```

except KeyboardInterrupt:
    pass
finally:
    node.timer_.cancel()
    node.destroy_node()
    if rclpy.ok():
        rclpy.shutdown()

if __name__ == "__main__":
    main()

```

注册入口点 (setup.py) :

```

entry_points={
    'console_scripts': [
        'random_spawner = turtle_spawn.random_spawner:main',
    ],
},

```

构建并运行:

```

cd ~/ros2_ws
colcon build --packages-select turtle_spawn
source install/setup.bash

```

终端 1 — Turtlesim (保持运行) :

```
ros2 run turtlesim turtlesim_node
```

终端 2 — 随机生成器:

```
ros2 run turtle_spawn random_spawner
```

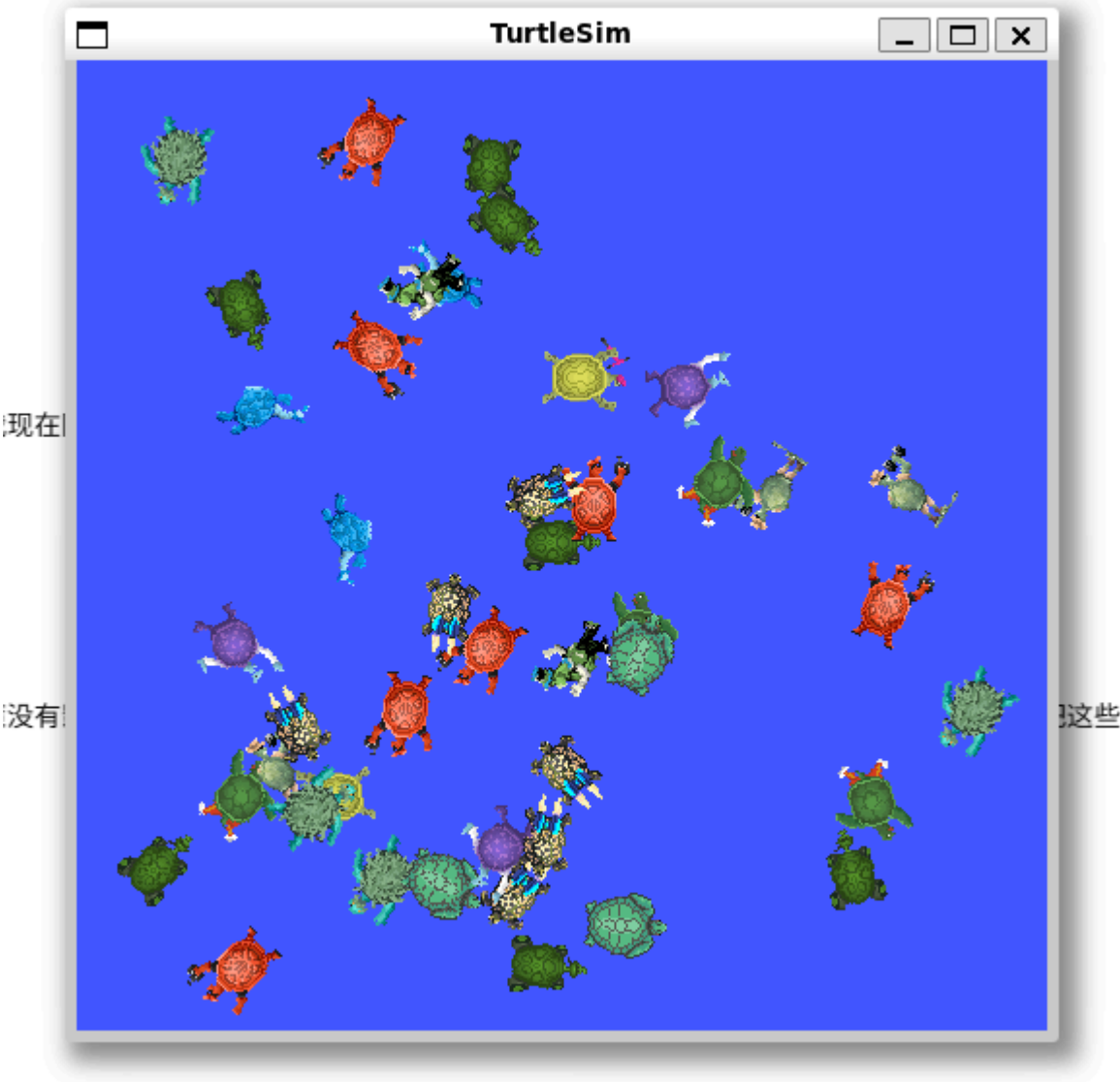
预期效果: 每 2 秒在 Turtlesim 窗口中随机位置出现一只新海龟, 终端打印生成信息:

```

[INFO] [random_spawner]: Spawned 'turtle2' at (3.7, 8.2)
[INFO] [random_spawner]: Spawned 'turtle3' at (6.1, 2.4)
[INFO] [random_spawner]: Spawned 'turtle4' at (9.0, 5.5)
...

```

变化的原因，但以后再修改代码也会更慢。



上图为本次课件配套源码手动运行 `turtlesim_node` 与 `random_spawner` 后截取的真实实验窗口画面。

实验验证清单

检查项	验证方法	预期结果
服务发现	<code>ros2 service list</code>	看到 <code>/spawn</code> 、 <code>/turtle1/set_pen</code> 等
手动生成海龟	<code>ros2 service call /spawn ...</code>	窗口出现新海龟
改变画笔颜色	<code>ros2 service call /turtle1/set_pen ...</code>	海龟轨迹颜色改变
Python Client	<code>ros2 run turtle_spawn random_spawner</code>	每 2 秒自动生成海龟
服务连通性	<code>ros2 service find turtlesim/srv/Spawn</code>	返回 <code>/spawn</code>

扩展练习 (选做)

1. **彩色海龟**: 在 `spawn_callback` 中根据海龟编号调用 `/turtleN/set_pen` 设置不同颜色
2. **计数限制**: 修改代码, 最多生成 5 只海龟后自动停止
3. **C++ 版本**: 将 `random_spawner.py` 改写为 C++ 节点, 使用 `turtlesim::srv::Spawn`

关键点

- Turtlesim 提供 `/spawn`、`/turtle1/set_pen` 等标准服务, 是练习 Service 通信的理想环境
 - `/spawn` 返回新海龟名称; `/<turtle_name>/set_pen` 的 Response 为空, 只需关注 Request
 - 每只海龟有独立的服务命名空间, 如 `/turtle2/set_pen`
 - Python Client 结合定时器 + `call_async` 实现周期性服务调用
 - 先命令行验证, 再编写代码, 是 Service 开发的最佳实践
-

本节总结

命令速查表

命令	说明
<code>ros2 service list</code>	列出所有活跃服务
<code>ros2 service list -t</code>	列出服务及其类型
<code>ros2 service type <service></code>	查看指定服务的类型
<code>ros2 service find <type></code>	按类型查找服务名
<code>ros2 service call <name> <type> "<data>"</code>	手动调用服务
<code>ros2 interface show <type></code>	查看服务/消息类型定义
<code>ros2 run py_srv add_two_ints_server</code>	运行 Python Service Server
<code>ros2 run py_srv add_two_ints_client_no_oop</code>	运行 Python Client (非 OOP)
<code>ros2 run py_srv add_two_ints_client</code>	运行 Python Client (OOP)
<code>ros2 run cpp_srv add_two_ints_server</code>	运行 C++ Service Server
<code>ros2 run cpp_srv add_two_ints_client_no_oop</code>	运行 C++ Client (非 OOP)
<code>ros2 run cpp_srv add_two_ints_client</code>	运行 C++ Client (OOP)
<code>ros2 run turtlesim turtlesim_node</code>	启动 Turtlesim 仿真器
<code>ros2 run turtle_spawn random_spawner</code>	运行随机海龟生成器
<code>ros2 service call /spawn turtlesim/srv/Spawn "{x: 5.0, y: 5.0, theta: 0.0, name: ''}"</code>	生成新海龟
<code>ros2 service call /turtle1/set_pen turtlesim/srv/SetPen "{r: 255, g: 0, b: 0, width: 3, 'off': 0}"</code>	设置画笔颜色

核心 API 对照表 (Python vs C++)

操作	Python (rclpy)	C++ (rclcpp)
导入服务类型	<code>from example_interfaces.srv import AddTwoInts</code>	<code>#include "example_interfaces/srv/add_two_ints.hpp"</code>
创建 Server	<code>create_service(AddTwoInts, "name", cb)</code>	<code>create_service<AddTwoInts>("name", cb)</code>
创建 Client	<code>create_client(AddTwoInts, "name")</code>	<code>create_client<AddTwoInts>("name")</code>
等待 Server	<code>client.wait_for_service(1.0)</code>	<code>client->wait_for_service(1s)</code>
构造请求	<code>req = AddTwoInts.Request()</code>	<code>auto req = std::make_shared<AddTwoInts::Request>()</code>
发送请求 (异步)	<code>client.call_async(request)</code>	<code>client->async_send_request(request)</code>
发送请求 (异步 + 回调)	<code>call_async(req) + add_done_callback(cb)</code>	<code>async_send_request(req, cb)</code>
同步等待响应	<code>spin_until_future_complete(node, future)</code>	<code>spin_until_future_complete(node, future)</code>
获取响应	<code>future.result()</code>	<code>future.get()</code>
Server 回调参数	<code>(request, response)</code>	<code>(Request::SharedPtr, Response::SharedPtr)</code>
访问请求字段	<code>request.a</code>	<code>request->a</code>
填充响应字段	<code>response.sum = ...</code>	<code>response->sum = ...</code>
绑定成员回调	直接传 <code>self.callback</code>	<code>std::bind(&Class::method, this, _1, _2)</code>
事件循环	<code>rclpy.spin(node)</code>	<code>rclcpp::spin(node)</code>

下一步

下一节 **2.4 动作 (Action) 通信** 中，我们将学习 ROS 2 的第三种通信机制——Action。与 Service 的"一次请求、一次响应"不同，Action 面向**长时间运行的任务**，提供**目标 (Goal)**、**反馈 (Feedback)** 和**结果 (Result)** 三段式通信。典型场景包括导航到目标点、机械臂抓取物体等需要数秒到数分钟才能完成的任务。

我们将学习如何定义 Action 接口文件，编写 Python 和 C++ 的 Action Server 与 Client，并掌握目标取消、进度反馈等高级特性。